

Reddit Pain Point Detector

Projet principal — Système ML de monitoring continu

Objectif

Construire un système de monitoring continu des besoins non satisfaits exprimés sur Reddit dans un domaine paramétrable. Le système ingère des données de manière incrémentale, recalcule l'analyse périodiquement, et expose en permanence une vision à jour via une API REST et un serveur MCP.

COLLECTE INCRÉMENTALE

Pipeline de collecte régulière des nouveaux contenus depuis la dernière exécution, sur des communautés ciblées dans un domaine paramétrable. Nettoyage, déduplication, horodatage et stockage structuré avec conservation des métadonnées.

→ *Quels critères permettent de distinguer un contenu porteur d'un pain point d'un bruit anecdotique ?*

GÉNÉRATION D'EMBEDDINGS

Transformation des nouveaux documents en représentations vectorielles et indexation dans un vector store. Seuls les nouveaux documents sont traités à chaque cycle — les embeddings existants sont conservés.

→ *Quelle stratégie de segmentation préserve la cohérence sémantique sans diluer le signal utile ?*

RECALCUL HEBDOMADAIRE DU CLUSTERING

Relance périodique de l'algorithme de clustering sur l'ensemble du corpus pour produire une vision à jour des thèmes. Labellisation automatique de chaque cluster. Archivage de l'état précédent pour permettre la comparaison dans le temps.

→ *Comment valider que les clusters reflètent des réalités business et non de simples proximités lexicales ?*

COUCHE AGENTIQUE

Agent qui enrichit les clusters identifiés, contextualise chaque thème avec des exemples verbatim issus des données, estime le volume et la récurrence, et produit un rapport structuré. Chaque étape de raisonnement est traçable et auditable.

→ *Comment garantir que l'agent s'appuie uniquement sur les données sources sans extrapoler ?*

ÉVALUATION & MLOPS

Évaluation indépendante de chaque brique sur un dataset de test construit manuellement. Comparaison d'au moins cinq configurations du pipeline avec métriques documentées. Traçabilité des runs et monitoring de la dérive dans le temps.

→ *Quelles métriques reflètent réellement l'utilité du système pour un utilisateur final ?*

DÉPLOIEMENT — DEUX INTERFACES, UN SEUL MOTEUR

API REST : endpoint documenté, containerisé, déployé publiquement, retournant l'état courant des pain points à tout moment. Serveur MCP : exposition des capacités du système comme outils appelables directement par un agent IA — état courant, évolution sur une période, détail d'un thème, exemples verbatim récents.

→ *Quels outils MCP ont une valeur réelle pour un agent qui interroge ce système ?*

Cycle de fonctionnement

Fréquent — collecte des nouveaux contenus, génération des embeddings, mise à jour du vector store.

Hebdomadaire — recalcul du clustering sur l'ensemble du corpus, relabellisation, archivage de l'état précédent, mise à jour de l'API et du serveur MCP.

Livrables

Pipeline d'ingestion incrémental

Fonctionnel, automatisé, reproductible.

API REST déployée

Documentée, containerisée, accessible publiquement.

Serveur MCP

Outils appelables par un agent IA, couvrant l'état courant et l'historique.

Comparaison de configurations

Au moins cinq runs documentés avec métriques calculées sur chaque brique.

Dataset de test annoté

Ground truth construit manuellement, scores calculés.

GitHub & write-up

README qui raconte une histoire technique, démo visuelle, article publié.

La valeur de ce projet tient à sa complétude et à sa tenue dans le temps. Un système qui ingère, analyse, évalue et expose de manière fiable sur plusieurs semaines est plus convaincant qu'un système sophistiqué qui s'arrête au notebook. Les choix techniques sont laissés ouverts — ce sont eux qui devront être défendus.